

1.4 Functions

Sue is working on a large project and is getting overwhelmed. How can she possibly keep all this code straight? To simplify her work, she decides to break the program into a collection of smaller, more manageable chunks. Each chunk can be individually designed, developed, and tested. Suddenly the problem seems much more manageable!

Objectives

By the end of this class, you will be able to:

- Create a function in C++.
- Pass data into a function using both pass-by-value and pass-by-reference.
- Be able to identify the scope of a variable in a program.

Prerequisites

Before reading this section, please make sure you are able to:

- Choose the best data-type to represent your data (Chapter 1.2).
- Declare a variable (Chapter 1.2).

Overview

A function is a small part of a larger program. Other terms (procedure, module, subroutine, subprogram, and method) mean nearly the same thing in the Computer Science context and we will use them interchangeably this semester.

There are two main ways to look at functions. The first is like a medical procedure. A medical procedure is a small set of tasks designed to accomplish a specific purpose. Typically these procedures are relatively isolated; they can be used in a wide variety of contexts or operations. Functions in C++ often follow this procedural model: breaking large programs into smaller ones.

The second way to look at functions is similar to how a mathematician looks at functions: an operation that converts input into output. The Cosine function is a great example: input in the form of radians or degrees is converted into a number between one and negative one. Frequently functions in C++ follow this model as programmers need to perform operations on data.

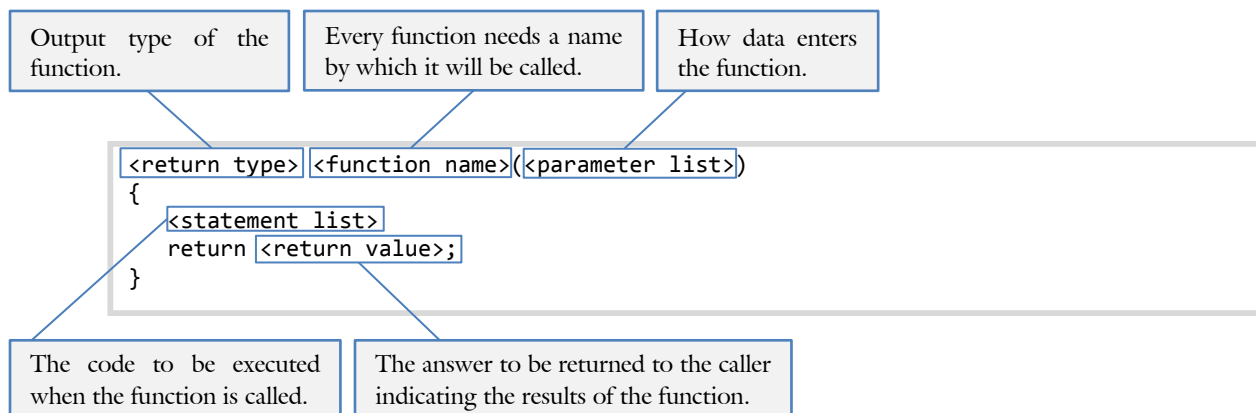
The syntax for both procedures and mathematical functions is the same in C++. The purpose of this chapter is to learn the syntax of functions so they can be used in our programs. All assignments, projects, and tests in this class will involve multiple functions from this time forward.

Function Syntax

There are two parts to function syntax: the syntax of declaring (or defining) a function, and the syntax of calling (or “running”) a function.

Declaring a Function

The syntax of a function is exactly the same as the syntax of `main()` because `main()` is a function!



Consider the following function to convert feet to meters:

```

/*****
 * CONVERT FEET TO METERS
 * Convert imperial feet to metric meters
 *****/
double convertFeetToMeters(double feet)
{
    double meters = feet * 0.3048;
    return meters;
}

```

Observe the function header. As the number of functions gets large in a program, it becomes increasingly important to have complete and concise function comment blocks.

Function names are typically verbs because functions do things. Similarly variable names are typically nouns because variables hold things. As with the function headers, strive to make the function names completely and concisely describe what the function does.

Finally, observe how one piece of information enters the function (`double feet`) and one piece of information leaves the function (`return meters;`). The input parameter (`feet`) is treated like any other variable inside the function.

Calling a Function

Calling a function is similar to looking up a footnote in the scriptures. The first step is to mark your current spot in the reading so you can return once the footnote has been read. The second step is to read the contents of the footnote. The third is to return back to your original spot in the reading. Observe that we can also jump to the Topical Guide or Bible Dictionary from the footnote. This requires us to remember our spot in the footnote as well as our spot in the scriptures. While humans can typically only remember one or two spots before their place is lost, computers can remember an extremely large number of places.

Computers follow the same algorithm when calling functions as we do when looking up a footnote:

```

{
    double heightFeet = 5.9;
    double heightMeters = convertFeetToMeters(heightFeet);
}

```

In this example, the user is converting his height in feet to the meters equivalent. To accomplish this, the function `convertFeetToMeters()` is called. This indicates the computer must stop working in the calling function and jump to the function `convertFeetToMeters()` much like a footnote in the scriptures indicates we should jump to the bottom of the page. After the computer has finished executing the code in `convertFeetToMeters()`, control returns to the calling function.

Example 1.4 – Simple Function Calling

Demo

This example will demonstrate how call a function and accept input from a function. There will be no parameter passing in this example.

Problem

Write a program to ask a simple question and receive a simple answer. This problem is inspired from one of the great literary works of our generation.

```
What is the meaning of life, the universe, and everything?  
The answer is: 42
```

The first function will return nothing. Hence, it will have the obvious name:

```
/* *****  
 * RETURN NOTHING  
 * This function will not return anything. Its only purpose is  
 * to display text on the screen. In this case, it will display  
 * one of the great questions of the universe  
 * ***** */  
void returnNothing()  
{  
    // display our profound question  
    cout << "What is the meaning of life, the universe, and everything?\n";  
  
    // send no information back to main()  
    return;  
}
```

The second function will return a single integer value back to the caller.

```
/* *****  
 * RETURN A VALUE  
 * This function, when called, will return a single integer value.  
 * ***** */  
int returnAValue()  
{  
    // did you guess what value we will be returning here?  
    return 42;  
}
```

The two functions can be called from main:

```
int main()  
{  
    // call the function asking the profound question  
    returnNothing();           // no data is sent to main()  
  
    // display the answer:  
    cout << "The answer is: "  
        << returnAValue()      // the return value of 42 is sent to COUT  
        << endl;  
  
    return 0;  
}
```

See Also

The complete solution is available at [1-4-simpleFunctionCalling.cpp](#) or:

```
/home/cs124/examples/1-4-simpleFunctionCalling.cpp
```

Solution

Example 1.4 – Prompt Function	
Demo	This example will demonstrate how create a simple prompt function. This function will display a message to the user asking him for information, receive the information using <code>cin</code> , and return the value through the function return mechanism.
Problem	Write a program to prompt the user for his age. The user's age will then be displayed. User input is <u>bold and underlined</u>. <pre>What is your age? <u>19</u> Your age is 19 years old.</pre>
Solution	The prompt function follows the “return a value” pattern from the previous example: <pre> /***** * GET AGE * Prompt the user for his age. First display a message stating * what information we hope the user will provide. Next receive * the user input. Finally, return the results to the caller. *****/ int getAge() { int age; // we need a variable to store the user input cout << "What is your age? "; // state what you want the user to give you cin >> age; // we need a variable to store the user input return age; // this sends data back to main() }</pre> Next we will create <code>main()</code> to test our function. <pre> /***** * MAIN * The whole purpose of main() is to test our getAge() function. *****/ int main() { // get the user input int age = getAge(); // store the data from getAge() in a variable // display the results cout << "Your age is " // note the space after "is" << age // the value from getAge() is stored here << " years old.\n"; // again a space before "year" return 0; // return "success" }</pre>
Challenge	As a challenge, try to add a new function to prompt for GPA. Note that this one will return a floating point number instead of an integer. What changes will you have to add to <code>main()</code> to test this function?
See Also	The complete solution is available at 1-4-promptFunction.cpp or: <pre>/home/cs124/examples/1-4-promptFunction.cpp</pre> 

Parameter Passing

Parameter passing is the process of sending data between functions. The programmer can send only one piece of data from the callee (the function responding to the function call) and the caller (the function issuing or initiating the function call). This data is sent through the return mechanism. However, the programmer can specify an unlimited amount of data to flow from the caller to the callee through the parameter passing mechanism.

Multiple Parameters

To specify more than one parameter to a function in C++, the programmer lists each parameter as a comma-separated list. For example, consider the scenario where the programmer is sending a row and column coordinate to a display function. The display function will need to accept two parameters.

```

/*****
 * DISPLAY COORDINATES
 * Display the row and column coordinates on the screen
 *****/
void displayCoordinates(int row, int column) // two parameters are expected
{
    cout << "("
          << row                      // the row parameter is the first passed
          << ", "
          << column                  // the column parameter is the second
          << ")\n";
    return;
}

```

For this function to be called, two values need to be provided.

```
displayCoordinates(5, 10);
```

Parameter match-up occurs by order, not by name. In other words, the first parameter sent to `displayCoordinates()` will always be sent to the `row` variable. The second parameter will always be sent to the `column`.

Note that the two parameters do not need to be of the same data-type.

```

/*****
 * computePay
 * Compute pay based on wage and number of hours worked
 *****/
double computePay(float wage, int hoursWorked)
{
    return (double)(wage * hoursWorked);
}

```

Common mistakes when working with parameters include:

- Passing the wrong number of parameters. For example, the function may expect two parameters but the programmer only supplied one:

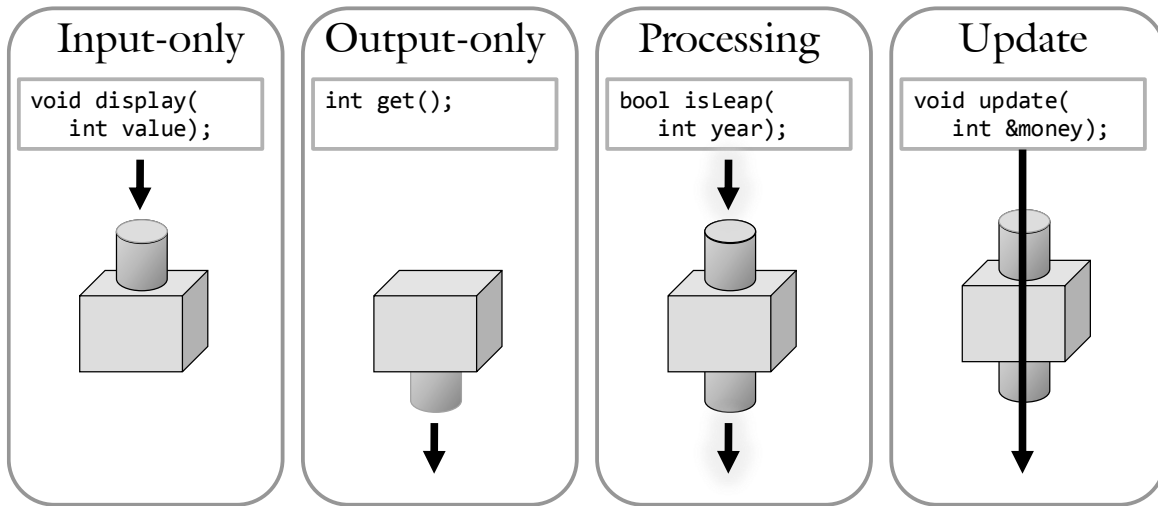
```
displayCoordinates(4); // two parameters expected. Where is the second?
```

- Getting the parameters crossed. For example, the function expects the first parameter to be `row` but the programmer supplied `column` instead:

```
displayCoordinates(column, row); // first parameter should be row, not column
```

Working with Parameters

There are four main ways to think of parameter passing in a C++ program:



Input Only: The first way involves data traveling one-way into a function. Observe how there is an input parameter (`int value`) but no return type (`void`). This is appropriate in those scenarios when you want a function to do something with the data (such as display it) but do not want to send any data back to the caller:

```
void display(int value);
```

Output Only: The second way occurs when data flows from a function, but not into it. An example would be a function prompting the user for information (such as `getIncome()` from Project 1). In this case, the parameter list is empty but there is a return value.

```
int get();
```

Processing: The third way occurs when a function converts data from one type to another. This model was followed in both our `computeSavings()` and `convertFeetToMeters()` examples. It is important to realize that you can have more than one input parameter (in the parentheses) but only one output parameter (the return mechanism).

```
bool isLeap(int year);
float add(float value1, float value2);
```

Update: The final way is when data is converted or updated in the function. This special case occurs when the input parameter and the return value are the same variable. In this case, we need a special indicator on the variable in the parameter list to specify that the variable is shared between the caller and the callee. We call this **call-by-reference**.

```
void update(int &money);
```

Example 1.4 – Compute Function	
Demo	This example will demonstrate how to send data to a function and receive data from a function. This follows the “Processing” model of information flow through a function.
Problem	Write a program to compute how much money to put in a savings account. The policy is “half the income after tithing is removed.” What is your allowance? <u>10.00</u> You need to deposit \$4.50
Solution	The function to compute savings takes income as input and returns the savings amount <pre>/* ***** * For a given amount of earned income, compute the amount to be saved * ***** */ int computeSavings(int centsIncome) { // first take care of tithing int centsTithing = centsIncome / 10; // D&C 119:4 centsIncome -= centsTithing; // remove tithing from the income // next compute the savings int centsSavings = centsIncome / 2; // savings are half the remaining return centsSavings; }</pre>
	This function will be called from main. It will provide the input centsIncome and present the results to the user through a cout statement. <pre>/* ***** * Prompt the user for his allowance and display the savings component * ***** */ int main() { // prompt the user for his allowance float dollarsAllowance; // a float for decimal #s cout << "What is your allowance? "; cin >> dollarsAllowance; // input is in dollars int centsAllowance = (int)dollarsAllowance * 100; // convert to cents // display how much is to be deposited int centsDeposit = computeSavings(centsAllowance); // call the function! cout << "You need to deposit \$" << (float)centsDeposit / 100.0 // convert back to dollars << endl; return 0; }</pre>
Challenge	As a challenge, create a function to convert a floating-point dollars amount (dollarsAllowance) into an integral cents amount (centsAllowance). Use the formula currently in main(): int centsAllowance = (int)dollarsAllowance * 100;
See Also	The complete solution is available at 1-4-computeFunction.cpp or: /home/cs124/examples/1-4-computeFunction.cpp 

In the preceding example, there are a few things to observe about the function `computeSavings()`. First, integers were chosen instead of floating point numbers. This is because, though `floats` can work with decimals, they are approximations and often yield unwieldy answers containing fractions of pennies! It is much cleaner to work with integers when dealing with money. To make sure this is obvious, include the units in the variable name.

Data is passed from `main()` into the function `computeSavings()` through the `()`s after the function name. In this case, the expression containing the variable `centsAllowance` is evaluated (to the value 2150 if the user typed 21.50). This value (not the variable!) is sent to the function `computeSavings()` where a new variable called `centsIncome` is created. This variable will be initialized with the value from the calling function (2150 in this case). It is important to realize that a copy of the data from `main()` is sent to the function through the parameter list; the variable itself is not sent! In other words, the variable `centsIncome` in `computeSavings()` can be changed without the variable `centsAllowance` in `main()` being changed. This is because they are different variables referring to different locations in memory!

When execution is in the function `computeSavings()`, only variables declared in that function can be used. This means that the statements in the function only have access to the variables `centsIncome`, `centsTithing`, and `centsSavings`. The variables from the caller (`dollarsAllowance`, `centsAllowance`, and `centsDeposit`) are not visible to `computeSavings()`. To pass data between the functions, parameters must be used

Pass-By-Reference

Pass-by-reference, otherwise known as “call-by-reference” is the process of indicating to the compiler that a given parameter variable is shared between the caller and the callee. We use the ampersand `&` to indicate the parameter is pass-by-reference.

Pass By Value	Pass By Reference
Pass-by-value makes a copy so two independent variables are created. Any change to the variable by the function will not affect the caller.	Pass-by-reference uses the same variable in the caller and the callee. Any change to the variable by the function will affect the caller.
<pre> /***** * Pass-by-value * No change to the caller *****/ void notChange(int number) { number++; } </pre>	<pre> /***** * Pass-by-reference * Will change the caller *****/ void change(int &number) { number++; } </pre>

We use pass-by-reference to enable a callee to send more than one piece of data back to the caller. Recall that the return mechanism only allows a single piece of data to be sent back to the caller. An example of this would be:

```

void getCoordinates(int &row, int &column)           // data is sent back by row and column
{
    cout << "Specify the coordinates (r c): ";
    cin  >> row >> column;
    return;                                         // no data is sent using return
}
  
```


Example 1.4 – Passing By Reference

Demo

This example will demonstrate how to pass no data to a function, how to use pass-by-value, and how to use pass-by-reference.

The first function does not pass any data into or out of the function:

```
/* *****  
 * PASS NOTHING: No information is being sent to the function  
 * ***** */  
void passNothing()  
{  
    // a different variable than the one in MAIN  
    int value;  
    value = 0;  
}
```

The second passes data into the function, so the function gets a copy of what the caller sent it. This is called pass-by-value:

```
/* *****  
 * PASS BY VALUE: One-way flow of information from MAIN to the function.  
 *               No data is being sent back to MAIN  
 * ***** */  
void passByValue(int value)  
{  
    // show the user what value was sent to the function  
    cout << "passByValue(" << value << ")\n";  
  
    // this is a copy of the variable in MAIN. This will not  
    // influence MAIN in any way:  
    value = 1;  
}
```

The final uses pass-by-reference. This means that both the caller and the callee share a variable. This relationship is indicated by the '&' symbol beside the parameter in the function:

```
/* *****  
 * PASS BY REFERENCE: Two-way flow of data between the functions. Changes to  
 *               REFERENCE will also influence the variable in MAIN  
 * ***** */  
void passByReference(int &reference)  
{  
    // show the user what value was sent to the function  
    cout << "passByReference(" << reference << ")\n";  
  
    // this will actually change MAIN because there was the &  
    // in the parameter  
    reference = 2;  
}
```

The only difference between `passByReference(int &reference)` and `passByValue(int value)` is the existence of the & beside the variable name. When the & is specified, then pass-by-reference is used.

See Also

The complete solution is available at [1-4-passByReference.cpp](#) or:

```
/home/cs124/examples/1-4-passByReference.cpp
```

Unit 1

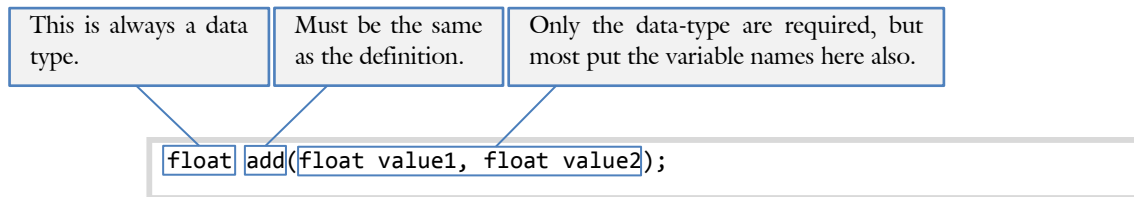
Solution

Prototypes

C++ programs are compiled linearly from the top of the file to the bottom. At the point in time when a given line of code is compiled, the compiler must know about all the variables and functions referenced in order for it to be compiled correctly. This means that the variables and functions must be defined above the line of code in which they are referenced.

One fallout of this model is that `main()` must be at the bottom of the file. This is required because any function referenced by `main()` must be defined by the time it is referenced in `main()`. As you may imagine, this can be inconvenient at times. Fortunately, C++ gives us a way to work around this constraint.

Prototypes are a way to give the compiler “heads-up” that a function will be defined later in the program. There are three required parts to a prototype: the return type, the function name, and the data-type of the parameters.



One nice thing about prototypes is that it allows us to put `main()` at the top of the program file, preceded by a list of all the functions that will appear later in the file.

```

#include <iostream>
using namespace std;

void displayInstructions();

/*****
int main()
{
    displayInstructions();

    // pause
    char letter;
    cin >> letter;
    return 0;
}

*****/
void displayInstructions()
{
    cout << "Please press the <enter>"
         << " key to quit the program"
         << endl;
    return;
}

```

Prototype of the function `displayInstructions()`. Note the absence of code; this is just an outline.

Here the function is called inside `main()` even though `displayInstructions()` has not been defined yet. Normally we need to define a function before we call it.

Now the function `displayInstructions()` is defined, we better have exactly the same name, return type, and parameter list as the prototype or the compiler will get grumpy.

Scope

A final topic essential to understanding how data passes between functions is Scope. Scope is the context in which a given variable is available for use. For example, if a variable is defined in one function, it cannot be referenced in another. The general rule of variable scope is the following:

A variable is only visible from the point where it is declared to the next closing curly brace }

Local Variables

The most common way to declare variables is in a function. This is called a “local variable” because the variable is local to (or restricted to) one function. Consider the following example:

```

/*****
 * PRINT NAME
 * Display the user's name. No data is shared with MAIN
 *****/
void printName()
{
    char name[256];          // Local variable only visible in the function printName
    cin >> name;
    cout << name << endl;    // last line of code where name is in scope
}

/*****
 * MAIN
 * Because there are no parameters being passed, there is
 * no communication between main() and printName()
 *****/
int main()
{
    printName();              // no variables are in scope here
    return 0;
}

```

Here name is a local variable. Its buffer is created when printName() is called. We know for a fact that it is not used or relevant outside printName().

IF Local

Though we have not learned about IF statements yet, consider the following code:

```

{
    int first = 20;
    int second = 10;

    if (first > second)
    {
        int temp = first;    // the variable temp is in scope from here...
        first = second;
        second = temp;       // ... to here. The next line has a } ending the scope
    }

    cout << first << ", "    // only first and second are "in scope" at this point
        << second << endl;
}

```

Here the variable temp is only relevant inside the IF statement. We know this because the variable *falls out of scope* once the } is reached after the statement “second = temp;”. Because the scope of temp is IF local, it is only visible inside the IF statement. Therefore, there is no possibility for side effects.

Blocks

A variable is only visible until program execution encounters the closing `}` in which it is defined. Note that you can introduce `{}`s at any point in the program. They are called blocks. Consider the following example:

```
{
    display();

    // pause
    {
        cout << "Press any key to continue";
        char something;
        cin.get(something);
    }
}
```

// the purpose of the {}s here are to limit scope
// only "in scope" for two lines of code

Since we are going to throw away something anyway and the value is irrelevant, we want to make sure that it is never used in a way different than is intended. The block ensures this.

Globals

A global variable is defined as a variable defined outside any function, usually at the top of a file.

```
#include <iostream>
using namespace std;

int input;                                // global variables are evil! Be careful

/*****
 * MAIN
 * Global variables are evil!
 *****/
int main()
{
    cout << "Enter your age: ";
    cin >> input;

    if (input > 25)
        cout << "Man you are old!\n";

    return 0;
}
```

These are very problematic because they are accessible by any function in the entire program. It therefore becomes exceedingly difficult to answer questions like:

- Is the variable initialized?
- Who set the variable last?
- Who will set the variable next?
- Who will be looking at this variable and depending on its value?

Unfortunately, these questions are not only exceedingly difficult to answer with global variables, but they are exceedingly important when trying to fix bugs. For this reason, global variables are banned for all classes in the BYU-Idaho Computer Science curriculum.

Problem 1

Write the C++ statements for the following:

$$c = 2\pi r$$

$$8 + 3 = x$$

$$e = mc^2$$

$$x = \frac{1}{2}y$$

Please see page 47 for a hint.

Problem 2

What is the value of c when the expression is evaluated:

```
int    f = 34;  
float c = (f - 32) * 5 / 9;
```

Answer:

Please see page 49 for a hint.

Problem 3

Write a function to display "Hello World". Call it hello()

Answer:

Please see page 60 for a hint.

Problem 4

Write a function to return a number. Call it get()

Answer:

Please see page 64 for a hint.

Problem 5

What is the output?

```
int two()
{
    return 3;
}

int main()
{
    int one  = 2;
    int three = two() + one;

    cout << three << endl;

    return 0;
}
```

Answer:

Please see page 59 for a hint.

Problem 6

What is the output?

```
void a()
{
    cout << "a";
    return;
}

void b()
{
    cout << "bb";
    return;
}

int main()
{
    a();
    b();
    a();

    return 0;
}
```

Answer:

Please see page 59 for a hint.

Problem 7

What is the output?

```
double a(double b, double c)
{
    return b - c;
}

int main()
{
    float x = a(4.0, 3.0);
    float y = a(7.0, 5.0);

    cout << a(x, y) << endl;

    return 0;
}
```

Answer:

Please see page 62 for a hint.

Problem 8

What is the output?

```
double add(double n1, double n2)
{
    return n1 + n2;
}

int main()
{
    double n3 = add(0.1, 0.2);
    double n4 = add(n3, add(0.3, 0.4));

    cout << n4 << endl;

    return 0;
}
```

Answer:

Please see page 62 for a hint.

Problem 9

What is the output?

```
void weird(int a, int &b)
{
    a = 1;
    b = 2;
}

int main()
{
    int a = 3;
    int b = 4;

    weird(a, b);

    cout << a * b << endl;
    return 0;
}
```

Answer:

Please see page 65 for a hint.

Problem 10

What is the output?

```
void setTrue(bool a)
{
    a = true;
    return;
}

int main()
{
    bool a = false;

    setTrue(a);

    cout << (int)a << endl;

    return 0;
}
```

Answer:

Please see page 65 for a hint.

Problem 11

What is the output?

```
int main()
{
    cout << a(b()) << endl;
    return 0;
}

int a(int value)
{
    return value * 2;
}

int b()
{
    return 3;
}
```

Answer:

Please see page 67 for a hint.

Problem 12

What is the output?

```
char value = 'a';

int main()
{
    char value = 'b';

    if (true)
    {
        char value = 'c';
    }

    cout << value << endl;

    return 0;
}
```

Answer:

Please see page 67 for a hint.

Assignment 1.4

You should start this assignment by copying the file `/home/cs124/assignments/assign14.cpp` to your directory:

```
cp /home/cs124/assignments/assign14.cpp assignment14.cpp
```

There are two functions that you will need to write:

Display Message

Please create a function called `questionPeter()`. The function should not return anything but instead display the following message:

```
Lord, how oft shall my brother sin against me, and I forgive him?
Till seven times?
```

Display Answer

The second function called `responseLord()` will return the Lord's response: $7 * 70$. This function will not display any output but rather return a value to the caller.

Main

`main()` is provided in the file `/home/cs124/assignments/assign14.cpp`:

```
/* *****
 * Main will not do much here. First it will display Peter's question,
 * then it will display the Lord's answer
 * ***** */
int main()
{
    // ask Peter's question
    questionPeter();

    // the first part of the Lord's response
    cout << "Jesus saith unto him, I say not unto thee, Until seven\n";
    cout << "times: but, Until " << responseLord() << ".\n";

    return 0;
}
```

Example

```
Lord, how oft shall my brother sin against me, and I forgive him?
Till seven times?
Jesus saith unto him, I say not unto thee, Until seven
times: but, Until 490.
```

Instructions

Please verify your solution against:

```
testBed cs124/assign14 assignment14.cpp
```

Don't forget to submit your assignment with the name "Assignment 14" in the header.

Please see page 60 for a hint.